

Projet Big Data

US Health Insurance Marketplace

Pipeline Medallion (Bronze / Silver / Gold)

Realise par : Nathan LAMTARA et Killian NGOG

Matiere : Big Data

Annee : 2025 - 2026

Technologies : Spark, Docker, MinIO, PostgreSQL, FastAPI, Streamlit

Volume de donnees : 250 000 lignes

Document de livrable

Table des matieres

1.	Presentation du projet	3
2.	Architecture technique	4
3.	Stack technique et services	5
4.	Donnees	6
5.	Pipeline Medallion	7
5.1	Couche Bronze - Ingestion	7
5.2	Couche Silver - Traitement	8
5.3	Couche Gold - Datamarts	9
6.	API REST (FastAPI + JWT)	11
7.	Dashboard Streamlit	12
8.	Orchestration Airflow	13
9.	Deploiement Docker	14
10.	Guide d'execution	15

1. Presentation du projet

Ce projet implemente un pipeline Big Data complet pour l'analyse du marche de l'assurance sante aux Etats-Unis. Il s'appuie sur l'architecture Medallion (Bronze / Silver / Gold) pour structurer le flux de donnees depuis les sources brutes jusqu'aux datamarts analytiques.

L'objectif est de repondre a trois questions metier fondamentales :

- Question 1 : Comment varie l'accessibilite financiere des plans selon les etats et niveaux ?
- Question 2 : Quelle est la structure du marche en termes de reseaux de soins et d'assureurs ?
- Question 3 : Comment se comparent les plans en termes de copays, exclusions et couverture ?

Le pipeline traite 250 000 lignes de donnees a travers 12 services Docker, avec un traitement distribue via Apache Spark, un stockage intermediaire dans MinIO (compatible S3), et une exposition via une API REST securisee et un dashboard interactif.

2. Architecture technique

L'architecture suit le modele Medallion, organise en trois couches de qualite croissante des donnees :

SOURCES	BRONZE	SILVER	GOLD	EXPOSITION
+-----+	+-----+	+-----+	+-----+	+-----+
MySQL				FastAPI
(plans)	MinIO	MinIO/Hive	PostgreSQL	REST+JWT
+-----+	(Parquet)	Spark	DM	+-----+
+-----+	s3a://	+ Jointures		+-----+
PostgreSQL	datalake/	+ Window)	- Affordability	Streamlit
(costs)	raw/		- Market Struct	Dashboard
+-----+	+-----+	+-----+	- Competitive	+-----+
			+-----+	
Orchestration : Apache Airflow (DAG quotidien)				

Le flux de donnees est le suivant :

- Bronze : Les sources MySQL et PostgreSQL sont lues via JDBC par Spark (Feeder).
- Silver : Les donnees brutes sont validees, jointes et enrichies (Processor).
- Gold : Trois datamarts sont construits et ecrits dans PostgreSQL Gold (Datamart).
- Exposition : Les datamarts sont exposes via une API REST FastAPI avec JWT et un dashboard Streamlit.

3. Stack technique et services

Composant	Technologie	Port
Source A - Plans	MySQL 8.0	3306
Source B - Couts	PostgreSQL 15	5432
Data Lake	MinIO (S3-compatible)	9000/9001
Traitement distribue	Apache Spark 3.5	8080/7077
Metastore	Apache Hive 3.1.3	9083
Base Gold (Datamarts)	PostgreSQL 15	5433
Orchestration	Apache Airflow 2.8.1	8082
API REST	FastAPI + JWT	8000
Visualisation	Streamlit + Plotly	8501
Conteneurisation	Docker Compose (12 services)	-

Justification des choix

- Spark : Choisi pour le traitement distribue, les jointures complexes et les fonctions de fenetre sur de gros volumes.
- MinIO : Compatible S3, leger, auto-heberge. Remplace HDFS dans un contexte conteneurise.
- Parquet : Stockage colonnes, compression, partitionnement natif. Ideal pour le data lake.
- PostgreSQL : Base relationnelle performante pour les datamarts finaux et les requetes analytiques.
- FastAPI : Framework Python asynchrone, documentation Swagger automatique, validation Pydantic.
- JWT : Token stateless, pas de session serveur, standard industriel pour les API REST.
- Streamlit : Prototypage rapide de dashboards interactifs en Python.

4. Donnees

Les donnees sont generees par le script data/generate_data.py qui produit 250 000 lignes reparties sur deux fichiers CSV :

Source A - Plans (MySQL)

Colonne	Type	Description
PlanID	VARCHAR	Identifiant unique du plan
IssuerName	VARCHAR	Nom de l'assureur
IssuerID	VARCHAR	Identifiant de l'assureur
PlanName	VARCHAR	Nom commercial du plan
PlanType	VARCHAR	Type de plan
NetworkType	VARCHAR	HMO, PPO, EPO ou POS
MetalLevel	VARCHAR	Bronze, Silver, Gold, Platinum, Catastrophic
StateCode	CHAR(2)	Code etat US (51 etats + DC)
CountyName	VARCHAR	Nom du comte
MarketCoverage	VARCHAR	Type de marche
IsActive	BOOLEAN	Plan actif ou non
PlanYear	INT	Annee du plan

Source B - Couts (PostgreSQL)

Colonne	Type	Description
PlanID	VARCHAR	Cle de jointure avec Source A
IndividualDeductible	DECIMAL	Franchise individuelle (\$)
FamilyDeductible	DECIMAL	Franchise familiale (\$)
IndividualOutOfPocketMax	DECIMAL	Maximum a charge individuel (\$)
FamilyOutOfPocketMax	DECIMAL	Maximum a charge familial (\$)
CopayPrimaryCare	DECIMAL	Copay soins primaires (\$)
CopaySpecialist	DECIMAL	Copay spécialiste (\$)
CopayER	DECIMAL	Copay urgences (\$)
CopayGenericDrug	DECIMAL	Copay medicament generique (\$)
CoinsuranceRate	DECIMAL	Taux de coassurance (0-1)
HSAEligible	BOOLEAN	Eligible HSA
BenefitCategory	VARCHAR	Categorie de prestation
BenefitCovered	BOOLEAN	Prestation couverte
Exclusions	VARCHAR	Exclusions de couverture

5. Pipeline Medallion

5.1 Couche Bronze - Ingestion (feeder.py)

Le Feeder lit les deux sources via JDBC (Spark DataFrame API) et écrit les données brutes en format Parquet dans MinIO, partitionnées par date d'ingestion.

Processus

- Lecture de la table plans depuis MySQL via le driver `com.mysql.cj.jdbc.Driver`
- Lecture de la table costs depuis PostgreSQL via `org.postgresql.Driver`
- Ecriture en Parquet dans `s3a://datalake/raw/plans/year=YYYY/month=MM/day=DD`
- Ecriture en Parquet dans `s3a://datalake/raw/costs/year=YYYY/month=MM/day=DD`

Resultat : 250 000 plans + 250 000 costs écrits dans MinIO.

5.2 Couche Silver - Traitement (processor.py)

Le Processor applique 7 regles de validation, effectue la jointure entre plans et costs, et enrichit les donnees avec des fonctions de fenetre Spark SQL.

Regles de validation

#	Regle	Description
1	PlanID unique/non-null	Suppression des doublons et valeurs nulles
2	StateCode valide	Verification parmi les 51 codes US (50 etats + DC)
3	Franchise valide	Deductible ≥ 0 et $\leq$ Out-of-Pocket Max
4	NetworkType valide	Valeurs autorisees : HMO, PPO, EPO, POS
5	IssuerName non-vide	Nom de l'assureur obligatoire
6	MetalLevel valide	Bronze, Silver, Gold, Platinum ou Catastrophic
7	PlanYear ≥ 2020	Exclusion des plans trop anciens

Jointure

Inner join sur PlanID entre les tables plans (validees) et costs (validees).

Fonctions de fenetre (Window Functions)

Fonction	Window	Description
rank()	StateCode / Deductible ASC	Rang par franchise dans chaque etat
avg()	StateCode	Franchise moyenne par etat
count()	StateCode	Nombre de plans par etat
count()	StateCode, NetworkType	Plans par etat et reseau
pct_network	Calcule	% de chaque type reseau dans l'etat
avg()	StateCode, MetalLevel	OOP max moyen par etat et metal
score	Formule	Deductible + OOP_Max * 0.3

Resultat : 250 000 lignes enrichies ecrites dans s3a://datalake/silver/plans_enriched/

5.3 Couche Gold - Datamarts (datamart.py)

Le job Datamart lit les données Silver et construit 3 datamarts analytiques via Spark SQL, puis les écrit dans PostgreSQL Gold via JDBC.

Datamart 1 : Accessibilité Financière

Repond a la question : comment varient les coûts selon les états et niveaux de plan ?

Colonne	Description
StateCode, MetalLevel	Dimensions de regroupement
num_plans	Nombre de plans distincts
avg_individual_deductible	Franchise individuelle moyenne
avg_individual_oop_max	Maximum à charge moyen
avg_affordability_score	Score d'accessibilité (calculé en Silver)
min/max_deductible	Plage de franchise

Volume : 255 lignes (51 états x 5 niveaux métal).

Datamart 2 : Structure du Marché

Repond a la question : quelle est la répartition des réseaux et assureurs ?

Colonne	Description
StateCode, NetworkType, IssuerName	Dimensions de regroupement
num_plans	Nombre de plans
metal_diversity	Nombre de niveaux métal couverts
network_share_pct	Part de marché du type réseau dans l'état
county_coverage	Nombre de comtés couverts

Volume : 8 160 lignes.

Datamart 3 : Compétitivité des Plans

Repond a la question : comment se comparent les plans sur les copays et exclusions ?

Colonne	Description
StateCode, MetalLevel, NetworkType	Dimensions de regroupement
BenefitCategory	Catégorie de prestation
avg_copay_primary/specialist/er/generic	Copays moyens par type de soin
avg_coinsurance_rate	Taux de coassurance moyen
num_excluded	Nombre de prestations exclues
num_hsa_eligible	Nombre de plans éligibles HSA

Volume : 10 192 lignes.

6. API REST (FastAPI + JWT)

L'API REST expose les 3 datamarts avec authentification JWT, pagination et filtres. Elle est documentee automatiquement via Swagger UI.

Authentication

L'API utilise OAuth2 avec des tokens JWT (JSON Web Token). Deux comptes de demo sont preconfigures : admin/admin123 (role admin) et analyst/analyst123 (role analyst). Le token expire apres 60 minutes.

Endpoints

Methode	Endpoint	Description
POST	/auth/login	Authentification, retourne un token JWT
GET	/health	Verification de l'etat de l'API et de la BDD
GET	/datamarts/affordability	DM1 pagine avec filtres state, metal_level
GET	/datamarts/affordability/stats	Statistiques agregees DM1
GET	/datamarts/market-structure	DM2 pagine avec filtres state, network, issuer
GET	/datamarts/market-structure/by-network	Repartition nationale par reseau
GET	/datamarts/competitiveness	DM3 pagine avec filtres multiples
GET	/datamarts/competitiveness/copay-summary	Resume copays par metal level

Securite

- Tous les endpoints /datamarts/* necessitent un token JWT valide (header Authorization: Bearer).
- Sans token ou avec un token invalide, l'API retourne une erreur 401 Unauthorized.
- CORS active pour permettre l'accès depuis le dashboard Streamlit.

Pagination

Chaque endpoint de datamart supporte les parametres page et page_size. La reponse inclut le total de resultats, le nombre de pages, et les items de la page courante.

7. Dashboard Streamlit

Le dashboard offre une visualisation interactive des 3 datamarts avec 12 graphiques Plotly, organisés en 4 onglets. Il se connecte directement à PostgreSQL Gold.

Graphiques

#	Type	Description
1	Carte choroplethe	Franchise individuelle moyenne par état
2	Barres groupées	Franchise vs OOP Max par niveau métal
3	Box plot	Distribution du score d'accessibilité par niveau
4	Barres horizontales	Top états les plus/moins chers
5	Donut	Repartition des plans par type de réseau
6	Barres	Diversité des niveaux métal par réseau
7	Barres horizontales	Top 15 assureurs par nombre de plans
8	Heatmap	Franchise moyenne par état et type de réseau
9	Barres groupées	Copays moyens par niveau de plan
10	Barres groupées	Taux de coassurance par niveau et réseau
11	Barres	% de plans éligibles HSA par niveau
12	Heatmap	Exclusions par état et catégorie de prestation

Filtres interactifs

- Sélection des états (multiselect)
- Sélection des niveaux métal (Bronze, Silver, Gold, Platinum, Catastrophic)
- Sélection des types de réseau (HMO, PPO, EPO, POS)

Tous les graphiques se mettent à jour dynamiquement selon les filtres sélectionnés.

8. Orchestration Airflow

Apache Airflow orchestre le pipeline via un DAG (Directed Acyclic Graph) qui enchaîne les 3 étapes dans l'ordre, avec exécution quotidienne à 6h UTC.

DAG insurance_pipeline

```
feeder_bronze  >>  processor_silver  >>  datamart_gold
  (Bronze)           (Silver)           (Gold)
```

```
Schedule : 0 6 * * * (quotidien à 6h UTC)
Retries   : 1 (délai 5 min)
Catchup   : False
```

Tache	Type	Action
feeder_bronze	BashOperator	spark-submit feeder.py (MySQL+PG -> MinIO)
processor_silver	BashOperator	spark-submit processor.py (Validation+Jointure)
datamart_gold	BashOperator	spark-submit datamart.py (Datamarts -> PG Gold)

Chaque tâche lance un spark-submit dans le conteneur Spark Master. Les dépendances garantissent que le Silver ne démarre qu'après le Bronze, et le Gold qu'après le Silver.

9. Deploiement Docker

L'ensemble du projet est conteneurisé avec Docker Compose. 12 services sont définis dans docker-compose.yml :

Service	Image / Build	Role
mysql	Build ./docker/mysql	Source A : plans d'assurance
postgres-source	Build ./docker/postgres	Source B : couts
minio	minio/minio:latest	Data lake S3-compatible
minio-init	minio/mc:latest	Creation des buckets
postgres-hive	postgres:13	Backend Hive Metastore
hive-metastore	Build ./docker/hive	Catalogue des tables
spark-master	Build ./docker/spark	Noeud maitre Spark
spark-worker	Build ./docker/spark	Noeud worker Spark
postgres-gold	postgres:15	Base des datamarts
airflow	apache/airflow:2.8.1	Orchestration DAG
api	Build ./api	API REST FastAPI
visualization	Build ./visualization	Dashboard Streamlit

Reseau

Tous les services sont connectés au réseau Docker 'bigdata' (bridge), permettant la communication inter-conteneurs par nom de service.

10. Guide d'execution

Prerequis : Docker et Docker Compose installes.

Lancement automatise

```
# 1. Cloner le projet
git clone <repo_url>
cd Projet-big-data

# 2. Lancer le script de demo complet
./run_demo.sh
```

Le script automatise toutes les etapes :

- Generation des donnees (250 000 lignes)
- Telechargement des JARs JDBC pour Spark
- Demarrage des 12 conteneurs Docker
- Attente de la disponibilite des services
- Execution du pipeline Spark (Bronze, Silver, Gold)
- Verification des resultats dans PostgreSQL Gold

Lancement manuel

```
# 1. Generer les donnees
python3 data/generate_data.py

# 2. Telecharger les JARs JDBC
bash spark/download_jars.sh

# 3. Demarrer l'infrastructure
docker compose up -d --build

# 4. Executer le pipeline Spark
# Bronze
docker exec spark-master spark-submit /opt/spark-jobs/feeder.py ...
# Silver
docker exec spark-master spark-submit /opt/spark-jobs/processor.py ...
# Gold
docker exec spark-master spark-submit /opt/spark-jobs/datamart.py ...
```

Acces aux interfaces

Interface	URL	Identifiants
Spark Master UI	http://localhost:8080	-
MinIO Console	http://localhost:9001	minioadmin / minioadmin
Airflow	http://localhost:8082	admin / admin

API Swagger	http://localhost:8000/docs	-
Dashboard	http://localhost:8501	-

Arret

```
docker compose down -v
```

Annexe : Structure du projet

```
.
|-- README.md
|-- docker-compose.yml          # 12 services Docker
|-- run_demo.sh                 # Script de demo automatise
|-- data/
|   |-- generate_data.py        # Generateur (250k lignes)
|   |-- csv/                    # Fichiers CSV generes
|-- docker/
|   |-- mysql/init/             # Schema + chargement MySQL
|   |-- postgres/init/         # Schema + chargement PostgreSQL
|   |-- hive/                   # Config Hive Metastore
|   |-- spark/                  # Dockerfile Spark + JARs
|-- spark/
|   |-- download_jars.sh        # Telechargement JARs JDBC
|   |-- jobs/
|       |-- feeder.py           # Bronze : Sources -> MinIO
|       |-- processor.py        # Silver : Validation + Jointures
|       |-- datamart.py         # Gold : Datamarts -> PostgreSQL
|-- airflow/
|   |-- dags/
|       |-- insurance_pipeline.py # DAG Airflow
|-- api/
|   |-- Dockerfile
|   |-- main.py                 # FastAPI + JWT + pagination
|   |-- requirements.txt
|-- visualization/
|   |-- Dockerfile
|   |-- app.py                  # Streamlit + 12 graphiques
|   |-- requirements.txt
```